

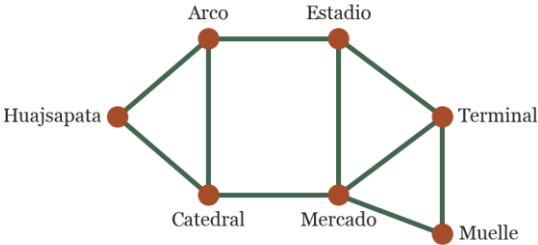
Prácticas de búsqueda: a mano y en JavaScript

Cinco temas, cinco ejercicios cada uno, del más fácil al más difícil. En cada uno: a) a mano, se llena la tabla; b) en código, se escribe la función en JavaScript. La página Practicas_busqueda.html tiene lo mismo y además comprueba las tablas y corre su código con pruebas. Profesor: Mg. Milton Vladimír Mamani Calisaya.

Nombre		Código	
--------	--	--------	--

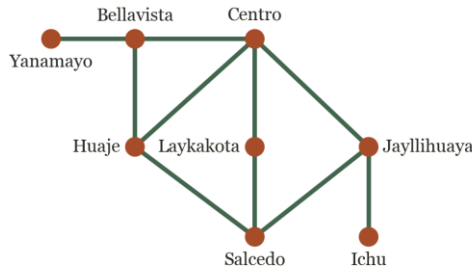
Los datos de los ejercicios

Simplificados: no son planos reales de Puno. Los vecinos van en el orden de la lista; en los mapas, en el orden arriba, abajo, izquierda, derecha. Columnas con letras y filas con números.



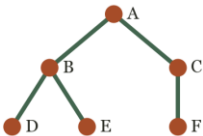
Siete lugares de Puno (simplificado: no es un plano real)

```
const grafoPuno = {
  Catedral: ["Arco", "Mercado", "Huajsapata"],
  Arco: ["Catedral", "Huajsapata", "Estadio"],
  Huajsapata: ["Catedral", "Arco"],
  Mercado: ["Catedral", "Muelle", "Estadio", "Terminal"],
  Muelle: ["Mercado", "Terminal"],
  Estadio: ["Arco", "Mercado", "Terminal"],
  Terminal: ["Muelle", "Estadio", "Mercado"],
};
```



Una red de combis entre barrios (simplificada: no son las rutas reales)

```
const redCombis = {
  Yanamayo: ["Bellavista"],
  Bellavista: ["Yanamayo", "Centro", "Huaje"],
  Centro: ["Bellavista", "Laykakota", "Huaje", "Jayllihuaya"],
  Huaje: ["Bellavista", "Centro", "Salcedo"],
  Laykakota: ["Centro", "Salcedo"],
  Salcedo: ["Huaje", "Laykakota", "Jayllihuaya"],
  Jayllihuaya: ["Centro", "Salcedo", "Ichu"],
  Ichu: ["Jayllihuaya"],
};
```



Un árbol pequeño

```
const arbol = {
  A: ["B", "C"],
  B: ["A", "D", "E"],
  C: ["A", "F"],
  D: ["B"],
  E: ["B"],
  F: ["C"],
};
```

mapa1 (ejercicios A4 y P5)

	A	B	C	D	E
1	S A1	B1	C1		E1
2	A2		C2	D2	E2
3	A3		M C3		E3
4	A4		C4		E4
5	A5	B5	C5	D5	E5

mapa2 (ejercicios A5)

	A	B	C	D	E
1	S A1	B1	C1	D1	
2	A2			D2	E2
3	A3	B3		D3	E3
4		B4		M D4	E4
5	A5	B5	C5	D5	E5

1. Búsqueda lineal

Recuerden: Se mira uno por uno, desde el primero (posición 0), hasta encontrarlo o hasta que se acabe la lista. No necesita que la lista esté ordenada. Si lo buscado no está, recién lo sabe al mirar el último.

L1 • Encontrar un número *muy fácil*

Busquen el **6** en la lista. Las posiciones se cuentan desde 0.

Datos: lista = [8, 3, 11, 6, 2, 9]

a) A mano. Una fila por comparación: qué número mira y si es el 6.

Comparación	Mira	¿Es el 6?
1		
2		
3		
4		

¿En qué posición está el 6? _____

¿Cuántas comparaciones hizo? _____

b) En código. Completen la función `buscarLineal` y pruébenla en `Practicas_busqueda.html`.

```
// Devuelve la posición de x en la lista (contando desde 0), o -1 si no está.
function buscarLineal(lista, x) {
  for (let i = 0; i < lista.length; i++) {
    // 1. Si lista[i] es igual a x (===), devuelve i

  }
  // 2. Si el for terminó, x no está: devuelve -1
}
```

L2 • Cuando no está *fácil*

Busquen el **10** en la lista. ¿Cuándo puede decir la búsqueda lineal que no está?

Datos: lista = [14, 7, 21, 3, 18]

a) A mano. Una fila por comparación.

Comparación	Mira	¿Es el 10?
1		
2		
3		
4		
5		

¿Cuántas comparaciones hizo? _____

¿Qué devuelve `buscarLineal(lista, 10)`? _____

b) En código. Completen la función `contarComparaciones` y pruébenla en `Practicas_busqueda.html`.

```
// Devuelve cuántas comparaciones hace la búsqueda lineal
// hasta encontrar x, o hasta que se acaba la lista.
function contarComparaciones(lista, x) {
  let comparaciones = 0;
  for (const valor of lista) {
    // 1. Suma 1 a comparaciones (cada vuelta es una comparación)

    // 2. Si valor es x, devuelve comparaciones

  }
  // 3. Si llegó aquí, miró todos: devuelve comparaciones
}
```

L3 • Todas las veces que aparece *intermedio*

En la lista de asistencia hay dos personas llamadas Ana. Ahora no basta con la primera: hay que encontrar **todas** las posiciones donde aparece "Ana".

Datos: asistencia = ["Rosa", "Juan", "Ana", "Luis", "Ana", "Pedro"]

a) A mano. Ahora no se para al encontrarla: se mira la lista entera.

Posición	Mira	¿Es "Ana"?
0		
1		
2		
3		
4		
5		

Posiciones donde está "Ana" (sepárenlas con comas) _____

¿Cuántas comparaciones hizo en total? _____

b) En código. Completen la función `todasLasPosiciones` y pruébenla en `Practicas_busqueda.html`.

```
// Devuelve una lista con TODAS las posiciones donde aparece x.
// Si no aparece nunca, devuelve una lista vacía [].
function todasLasPosiciones(lista, x) {
  const posiciones = [];
  // Recorran la lista con un for. Cuando lista[i] sea x,
  // agreguen i al final con posiciones.push(i). No hagan return dentro del for.

  return posiciones;
}
```

L4 • La noche más fría *intermedio*

Las temperaturas mínimas de una semana en Puno, en °C (datos de ejemplo). Busquen la posición de la **más baja**. Si se repite, vale la **primera** vez que aparece.

Aquí no hay atajo: hasta mirar el último no se puede saber cuál es la menor.

Datos: minimas = [-2, -5, -1, -6, -3, -6, -4]

a) A mano. En cada posición: qué valor mira y cuál es la menor hasta ese momento.

Posición	Mira	Menor hasta ahora
0		
1		
2		
3		
4		
5		
6		

b) En código. Completen la función `posicionDelMenor` y pruébenla en `Practicas_busqueda.html`.

```
// Devuelve la posición del valor más pequeño de la lista.
// Si el menor se repite, devuelve la PRIMERA posición.
function posicionDelMenor(lista) {
  let pos = 0; // por ahora, el menor es el primero
  // Recorran desde i = 1 hasta el final.
  // Si lista[i] es menor que lista[pos], ahora el menor está en i.

  return pos;
}
```

¿Qué posición devuelve? _____

¿Cuántos valores tuvo que mirar? _____

L5 • Buscar por código *reto*

La lista guarda objetos con código y apellido, sin ningún orden. Busquen el código **"241130"** y devuelvan su apellido. Si el código no está, la función devuelve `null`.

Datos: registros: 241122 Huanca • 241104 Apaza • 241148 Ticona • 241130 Mamani • 241117 Condori • 241141 Quispe

a) A mano. Primero el código **"241130"**. Si sobran filas, déjenlas en blanco.

Comparación	Mira el código	¿Coincide?
1		
2		
3		
4		
5		
6		

¿Qué devuelve para **"241130"**? _____

¿Cuántas comparaciones necesita para decir que **"241150"** no está? _____

b) En código. Completen la función `buscarPorCodigo` y pruébenla en `Practicas_busqueda.html`.

```
// registros es una lista de objetos como { codigo: "241122", nombre: "Huanca" }.
// Devuelve el nombre del registro que tiene ese código, o null si no está.
// Para leer el código del registro r: r.codigo
function buscarPorCodigo(registros, codigo) {

}
```

2. Búsqueda binaria

Recuerden: La lista tiene que estar ordenada. Se trabaja con posiciones: *inicio*, *fin* y *medio*. $\text{medio} = \text{Math.floor}((\text{inicio} + \text{fin}) / 2)$, es decir, sin decimales, redondeando hacia abajo. Si lo buscado es mayor que `lista[medio]`: $\text{inicio} = \text{medio} + 1$. Si es menor: $\text{fin} = \text{medio} - 1$. Para cuando lo encuentra, o cuando *inicio* pasa a *fin* ($\text{inicio} > \text{fin}$): entonces no está.

B1 • Partir a la mitad *muy fácil*

Busquen el **21** en la lista ordenada. El paso 1 ya está hecho: sigan desde el paso 2.

Datos: lista = [2, 4, 7, 10, 13, 18, 21, 25, 30]

a) A mano. En cada paso: *inicio*, *fin*, *medio*, el valor que mira y qué decide.

Paso	inicio	fin	medio	Mira lista[medio]	Decide
1	0	8	4	13	mayor: inicio = medio + 1
2					

Decide: «mayor» ($\text{inicio} = \text{medio} + 1$), «menor» ($\text{fin} = \text{medio} - 1$) o «lo encontró».

¿En qué posición está el 21? _____

¿Cuántas comparaciones hizo? (la lineal habría hecho 7) _____

b) En código. Completen la función `busquedaBinaria` y pruébenla en `Practicass_busqueda.html`.

```
// Devuelve la posición de x en la lista ORDENADA, o -1 si no está.
// Completen los tres espacios ____
function busquedaBinaria(lista, x) {
  let inicio = 0;
  let fin = lista.length - 1;
  while (inicio <= fin) {
    const medio = ____;           // 1. (inicio + fin) / 2 sin decimales: Math.floor(...)
    if (lista[medio] === x) {
      return medio;
    } else if (lista[medio] < x) {
      inicio = ____;              // 2. x es mayor: descarta la mitad izquierda
    } else {
      fin = ____;                 // 3. x es menor: descarta la mitad derecha
    }
  }
  return -1;
}
```

B2 • Cuando no está *fácil*

Busquen el **10**. No está: miren con qué valores de *inicio* y *fin* se detiene. Si sobra una fila, déjenla en blanco.

Datos: lista = [1, 3, 6, 9, 12, 15, 20, 24]

a) A mano. Un paso por fila.

Paso	inicio	fin	medio	Mira lista[medio]	Decide
1					
2					
3					
4					

Decide: «mayor» ($\text{inicio} = \text{medio} + 1$), «menor» ($\text{fin} = \text{medio} - 1$) o «lo encontró».

¿Cuántas comparaciones hizo? _____

Al detenerse, ¿cuánto vale *inicio*? _____

Al detenerse, ¿cuánto vale *fin*? _____

b) En código. Completen la función `contarPasosBinaria` y pruébenla en `Practicass_busqueda.html`.

```
// Hace la búsqueda binaria, pero devuelve CUÁNTAS veces miró
// un elemento (lo encuentre o no).
function contarPasosBinaria(lista, x) {
  let inicio = 0;
  let fin = lista.length - 1;
  let pasos = 0;
  while (inicio <= fin) {
    const medio = Math.floor((inicio + fin) / 2);
    // 1. Suma 1 a pasos
    // 2. Si lista[medio] es x, devuelve pasos
    // 3. Si es menor que x: inicio = medio + 1; si es mayor: fin = medio - 1

  }
  return pasos;
}
```

B3 · Palabras en orden alfabético *intermedio*

La binaria sirve con cualquier cosa que tenga orden. En JavaScript, "boga" < "ispi" es true: los textos se comparan en orden alfabético. Busquen **"suche"**.

Datos: palabras = ["alpaca", "boga", "carachi", "ispi", "llama", "pejerrey", "puma", "suche", "trucha"]

a) A mano. Igual que con números: «mayor» quiere decir que va después en el alfabeto. Si sobra una fila, déjenla en blanco.

Paso	inicio	fin	medio	Mira lista[medio]	Decide
1					
2					
3					
4					

Decide: «mayor» (inicio = medio + 1), «menor» (fin = medio - 1) o «lo encontró».

¿En qué posición está "suche"? _____

¿Cuántas comparaciones hace para concluir que "condor" no está? _____

b) En código. Completen la función buscarPalabra y pruébenla en Practicas_busqueda.html.

```
// Búsqueda binaria en una lista de palabras ordenada.  
// Devuelve la posición de palabra, o -1 si no está.  
// Es la misma búsqueda de los ejercicios anteriores: < y > también comparan textos.  
function buscarPalabra(lista, palabra) {  
  let inicio = 0;  
  let fin = lista.length - 1;  
  // mientras quede algo por mirar (inicio <= fin):  
  //  calculen medio, comparen lista[medio] con palabra y descarten una mitad  
  
  return -1;  
}
```

B4 · Lo que va mirando *intermedio*

Una lista de 15 números ordenados. Busquen el **48** y anoten qué valores mira. Si sobra una fila, déjenla en blanco.

Datos: lista = [3, 8, 12, 17, 21, 26, 30, 34, 39, 43, 48, 52, 57, 61, 66]

a) A mano. Un paso por fila.

Paso	inicio	fin	medio	Mira lista[medio]	Decide
1					
2					
3					
4					
5					

Decide: «mayor» (inicio = medio + 1), «menor» (fin = medio - 1) o «lo encontró».

Los valores que mira, en orden (separados por comas) _____

Si buscan el 70, que no está, ¿cuántas comparaciones hace? _____

b) En código. Completen la función binariaConTraza y pruébenla en Practicas_busqueda.html.

```
// Hace la búsqueda binaria y devuelve la lista de los valores que miró  
// (cada lista[medio]), en orden. Si lo encuentra, el último es x.  
function binariaConTraza(lista, x) {  
  const miradas = [];  
  // escriban aquí la búsqueda binaria; en cada vuelta: miradas.push(lista[medio])  
  
  return miradas;  
}
```

B5 • Adivina el número *reto*

Alguien piensa un número del **1 al 100** y solo contesta «es más», «es menos» o «¡acertó!». La computadora dice siempre el del medio. El número secreto es **73**.

Aquí no hay lista: inicio y fin son directamente los números (empiezan en 1 y 100) y el medio es el intento.

a) A mano. *Un intento por fila. Si sobra una fila, déjenla en blanco.*

Intento	inicio	fin	Dice (medio)	Le contestan
1				
2				
3				
4				
5				
6				
7				

Le contestan: «es más» (inicio = medio + 1), «es menos» (fin = medio - 1) o «¡acertó!».

¿Cuántos intentos necesitó para el 73? _____

Con este método, ¿cuántos intentos como máximo para cualquier número del 1 al 100? _____

b) En código. *Completen la función adivinar y pruébenla en Practicas_busqueda.html.*

```
// Devuelve la lista de intentos que hace la computadora para adivinar
// el número secreto, entre 1 y 100, diciendo siempre el del medio.
function adivinar(secreto) {

}
```

3. Grafos

Recuerden: *Un grafo son nodos unidos por aristas. Dos nodos unidos por una arista son vecinos. En JavaScript se guarda como un objeto: cada clave es un nodo y su valor es la lista de sus vecinos, en orden. grafo["Mercado"] da los vecinos de Mercado. Como las aristas van en los dos sentidos, si A tiene a B en su lista, B tiene a A en la suya.*

G1 • Los vecinos *muy fácil*

Miren el grafo de los siete lugares y su lista de adyacencia.

Datos: grafoPuno (página 1)

a) A mano. *Los vecinos en el orden en que aparecen en la lista. Pueden escribir los nombres completos o sus tres primeras letras.*

Nodo	Vecinos, en orden
Mercado	
Huajsapata	
Estadio	

¿Cuántos nodos tiene el grafo? _____

¿Qué escribe console.log(grafoPuno["Muelle"])? _____

b) En código. *Completen la función vecinos y pruébenla en Practicas_busqueda.html.*

```
// Devuelve la lista de vecinos de nodo.
// Si el nodo no está en el grafo, devuelve una lista vacía [].
// Completen los dos espacios ____
function vecinos(grafo, nodo) {
  if (____) {           // 1. ¿nodo está en el grafo? Pista: nodo in grafo
    return ____;        // 2. su lista de vecinos
  }
  return [];
}
```

G2 • Grados y aristas *fácil*

El **grado** de un nodo es cuántos vecinos tiene. Cada arista se cuenta dos veces en la lista (una en cada extremo), así que el número de aristas es la suma de los grados dividida entre 2.

Datos: grafoPuno (página 1)

a) A mano. El grado de cada nodo.

Nodo	Grado
Catedral	
Arco	
Huajsapata	
Mercado	
Muelle	
Estadio	
Terminal	

b) En código. Completen la función contarAristas y pruébenla en Practicas_busqueda.html.

```
// Devuelve cuántas aristas tiene el grafo.
function contarAristas(grafo) {
  let suma = 0;
  for (const nodo in grafo) {      // recorre las claves: los nombres de los nodos
    // 1. suma el grado de nodo (cuántos vecinos tiene: .length)

  }
  return ____;                    // 2. cada arista se contó dos veces
}
```

Suma de los grados

¿Cuántas aristas tiene el grafo? _____

G3 • ¿Hay arista? *intermedio*

Hay arista entre a y b si b aparece en la lista de vecinos de a. Contesten para cada par.

Datos: grafoPuno (página 1)

a) A mano. Miren la lista de adyacencia: ¿b está en la lista de a?

a	b	¿Hay arista?
Catedral	Arco	
Huajsapata	Mercado	
Muelle	Terminal	
Estadio	Catedral	
Terminal	Mercado	

Si existeArista(g, "Arco", "Estadio") da true, ¿qué da existeArista(g, "Estadio", "Arco")? _____

b) En código. Completen la función existeArista y pruébenla en Practicas_busqueda.html.

```
// Devuelve true si hay una arista entre a y b, y false si no.
// Si a no está en el grafo, devuelve false.
// Pista: una lista tiene el método .includes(valor), que da true o false.
function existeArista(grafo, a, b) {

}

}
```

G4 • El paradero con más conexiones *intermedio*

En la red de combis, ¿qué paradero tiene más vecinos? Se recorren los nodos en el orden del objeto y se recuerda el mejor hasta ahora. Si hay empate, gana el **primero** que apareció.

Datos: redCombis (página 1)

a) A mano. En cada fila: el grado del nodo y cuál es el mejor hasta ese momento.

Nodo	Grado	Mejor hasta ahora
Yanamayo		
Bellavista		
Centro		
Huaje		
Laykakota		
Salcedo		
Jayllihuaya		
Ichu		

¿Qué devuelve nodoConMasVecinos(redCombis)? _____

b) En código. Completen la función nodoConMasVecinos y pruébenla en Practicas_busqueda.html.

```
// Devuelve el nodo con más vecinos. Si hay empate, el que aparece primero.
// Es como buscar el menor de una lista (ejercicio L4), pero con el grado.
function nodoConMasVecinos(grafo) {
```

```

let mejor = null;
for (const nodo in grafo) {
  // si mejor es null, o nodo tiene MÁS vecinos que mejor, ahora mejor = nodo

}
return mejor;
}

```

G5 • ¿Es un camino? *reto*

Un camino es una lista de nodos donde cada uno es vecino del siguiente. Revisen cada tramo (de un nodo al siguiente) en la red de combis.

Datos: redCombis (página 1)

a) A mano. Si no es camino, escriban el primer tramo que falla, por ejemplo «Centro, Ichu». Si es camino, dejen esa celda en blanco.

Camino propuesto	¿Es camino?	Primer tramo que falla
Yanamayo → Bellavista → Centro → Laykakota		
Bellavista → Laykakota → Salcedo		
Ichu → Jayllihuaya → Centro → Huaje → Bellavista		
Centro → Salcedo → Jayllihuaya		

b) En código. Completen la función verificarCamino y pruébenla en Practicas_busqueda.html.

```

// Devuelve true si la lista camino es un camino válido en el grafo:
// todos los nodos existen y cada uno es vecino del siguiente.
// Un camino vacío [] no es válido. Un camino de un solo nodo existente sí.
function verificarCamino(grafo, camino) {

}

```

4. Amplitud (BFS)

Recuerden: La frontera es una COLA: sale el primero (el de la izquierda) y los nuevos entran al final. En JavaScript: frontera.shift() y frontera.push(...). 1. Sale uno. 2. Si ya está en visitados, se tacha y sale otro. 3. Si es la meta, termina. 4. Si no, va a visitados y entran sus vecinos no visitados, en el orden de la lista. Un nodo va a visitados cuando SALE de la frontera, no cuando entra. Por eso un nodo puede estar dos veces en la frontera. Las funciones devuelven { camino, orden }: el camino del inicio a la meta y el orden en que los nodos salieron a visitados.

A1 • Recorrer un árbol por niveles *muy fácil*

Recorran todo el árbol por amplitud desde A, sin meta: termina cuando la frontera queda vacía. La fila 1 ya está hecha.

Datos: arbol (página 1)

a) A mano. Frontera y visitados después de cada salida. Cuando la frontera quede vacía, escriban «vacía».

N.º	Sale	Frontera después	Visitados
1	A	B, C	A
2			
3			
4			
5			
6			

Orden en que salen (separados por comas) _____

b) En código. Completen la función recorridoAmplitud y pruébenla en Practicas_busqueda.html.

```

// Recorre TODO el grafo por amplitud desde inicio.
// Devuelve el orden en que los nodos salen de la frontera y van a visitados.
// Completen los dos espacios ____
function recorridoAmplitud(grafo, inicio) {
  const frontera = [inicio];      // la cola
  const visitados = [];
  while (frontera.length > 0) {
    const nodo = frontera.____(); // 1. sale el PRIMERO de la cola
    if (visitados.includes(nodo)) {
      continue;                  // repetido: se tacha
    }
    visitados.push(nodo);
    for (const vecino of grafo[nodo]) {
      if (!visitados.includes(vecino)) {
        frontera.____(vecino);    // 2. entra al FINAL de la cola
      }
    }
  }
  return visitados;
}

```

}

A2 • De Arco a Mercado *fácil*

Amplitud desde **Arco** hasta **Mercado**. Cada nodo que entra a la frontera recuerda quién lo trajo: con eso se reconstruye el camino al final.

Datos: grafoPuno (página 1)

a) A mano. Escriban los nombres completos o sus tres primeras letras. Si sale uno que ya está en visitados, escriban su nombre: esa salida se tacha y visitados no cambia. En la fila de la meta, en «Frontera después» va lo que queda (o «meta»).

N.º	Sale	Frontera después	Visitados
1			
2			
3			
4			
5			

Camino que encuentra (separado por comas) _____

¿Cuántas aristas tiene ese camino? _____

b) En código. Completen la función `bfs` y pruébenla en `Practicass_busqueda.html`.

```
// Amplitud desde inicio hasta meta.
// Devuelve { camino, orden }; si no llega, camino es null.
// Completen los tres espacios ____
function bfs(grafo, inicio, meta) {
  const frontera = [[inicio, null]]; // la cola: pares [nodo, quién lo trajo]
  const visitados = [];
  const padre = {};
  while (frontera.length > 0) {
    const [nodo, previo] = frontera.shift();
    if (visitados.includes(nodo)) {
      continue;
    }
    visitados.push(nodo);
    padre[nodo] = previo; // quién lo trajo
    if (nodo === ____ ) { // 1. ¿llegó?
      return { camino: reconstruir(padre, meta), orden: visitados };
    }
    for (const vecino of grafo[nodo]) {
      if (!visitados.includes(vecino)) {
        frontera.push([vecino, ____]); // 2. ¿quién lo trajo?
      }
    }
  }
  return { camino: null, orden: visitados };
}

// Arma el camino de atrás hacia adelante: meta, su padre, el padre de ese...
function reconstruir(padre, meta) {
  const camino = [meta];
  while (padre[camino[0]] !== null) {
    camino.unshift(____); // 3. el padre del primero, al inicio
  }
  return camino;
}
```

A3 • Con repetidos: de Laykakota a Yanamayo *intermedio*

En la red de combis hay ciclos, así que algunos nodos entran dos veces a la frontera. Amplitud desde **Laykakota** hasta **Yanamayo**.

En el código ya no hay espacios: escriban la función entera. Pueden copiar reconstruir del ejercicio A2.

Datos: redCombis (página 1)

a) A mano. Escriban los nombres completos o sus tres primeras letras. Si sale uno que ya está en visitados, escriban su nombre: esa salida se tacha y visitados no cambia. En la fila de la meta, en «Frontera después» va lo que queda (o «meta»). Si sobran filas, déjenlas en blanco.

N.º	Sale	Frontera después	Visitados
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Camino que encuentra _____

¿Cuántas salidas se tacharon? _____

b) En código. Completen la función bfs y pruébenla en Practicas_busqueda.html.

```
// Amplitud desde inicio hasta meta. Devuelve { camino, orden }.
// orden: los nodos que fueron a visitados, en orden (los tachados no cuentan).
// Si no llega a la meta: { camino: null, orden: ... }
//
// Las cuatro reglas:
// 1. sale el primero de la cola (shift)
// 2. si ya está en visitados, se tacha (continue)
// 3. va a visitados; si es la meta, termina
// 4. entran al final sus vecinos no visitados, cada uno con quién lo trajo
function bfs(grafo, inicio, meta) {

}

function reconstruir(padre, meta) {
  // cópienla del ejercicio A2
}
```

A4 • El mapa como grafo *intermedio*

Un mapa de 5×5 : # es muro, . es libre, S es el inicio y M la meta. Las columnas son letras (A a E) y las filas, números (1 a 5): S está en A1.

Cada casilla libre es un nodo y sus vecinos son las casillas libres de al lado, en el orden **arriba, abajo, izquierda, derecha**. Hagan amplitud de S (A1) a M (C3).

Datos: mapa1 (página 1)

a) A mano. Las casillas por su nombre: A1, B1... Vecinos siempre en el orden arriba, abajo, izquierda, derecha.

N.º	Sale	Frontera después	Visitados
1			
2			
3			
4			
5			
6			
7			
8			
9			

Camino de A1 a C3 _____

¿Cuántos pasos tiene? (aristas) _____

b) En código. Completen la función mapaAGrafo y pruébenla en Practicas_busqueda.html.

```
// Convierte un mapa (lista de textos) en un grafo: { "A1": [vecinos], ... }
// Solo las casillas libres son nodos (todo lo que no es "#").
```

```
// Vecinos en orden: arriba, abajo, izquierda, derecha; sin salirse del mapa.
// La casilla de la fila f y la columna c (contando desde 0) se llama
// LETRAS[c] + (f + 1): la fila 0, columna 2 es "C1".
const LETRAS = "ABCDEFGHIJ";

function mapaAGrafo(mapa) {
  const grafo = {};
  for (let f = 0; f < mapa.length; f++) {
    for (let c = 0; c < mapa[f].length; c++) {
      if (mapa[f][c] === "#") continue;
      const vecinos = [];
      // revisen las cuatro direcciones en orden: [-1, 0], [1, 0], [0, -1], [0, 1]
      // y agreguen el nombre de cada casilla que exista y no sea "#"

      grafo[LETRAS[c] + (f + 1)] = vecinos;
    }
  }
  return grafo;
}
```

A5 • Pasos mínimos *reto*

Amplitud encuentra el camino con menos pasos. En este otro mapa, escriban en cada casilla libre a cuántos pasos está de S. S vale 0; sus vecinos, 1; los vecinos de esos, 2; y así, por niveles.

Datos: mapa2 (página 1)

a) A mano. *Un número en cada casilla libre. Los muros ya están marcados.*

	A	B	C	D	E
1	0				
2					
3					
4					
5					

¿A cuántos pasos está M? _____

b) En código. *Completen la función pasosMinimos y pruébenla en Practicas_busqueda.html.*

```
// Devuelve el menor número de pasos de S a M en el mapa,
// o -1 si no se puede llegar.
// Pueden usar su mapaAGrafo (ejercicio A4) y su bfs (A3): los pasos son
// la cantidad de aristas del camino, es decir, camino.length - 1.
function pasosMinimos(mapa) {

}
```

5. Profundidad (DFS)

Recuerden: La frontera es una PILA: sale el último que entró (el de la derecha). En JavaScript: `frontera.pop()` y `frontera.push(...)`. Las mismas cuatro reglas de amplitud. Lo único que cambia es por dónde sale. Los vecinos entran en el orden de la lista, así que sale primero el ÚLTIMO vecino de la lista. Sin la lista de visitados, profundidad puede dar vueltas para siempre en un grafo con ciclos.

P1 • El mismo árbol, con pila *muy fácil*

El mismo árbol del ejercicio A1, ahora con una pila. Recorran todo desde A. La fila 1 ya está hecha.

Datos: arbol (página 1)

a) A mano. *La pila se escribe con el tope a la derecha: sale el de la derecha. Cuando quede vacía, escriban «vacía».*

N.º	Sale	Pila después	Visitados
1	A	B, C	A
2			
3			
4			
5			
6			

Orden en que salen _____

b) En código. *Completen la función recorridoProfundidad y pruébenla en Practicas_busqueda.html.*

```
// Recorre TODO el grafo por profundidad desde inicio.
// Devuelve el orden en que los nodos salen de la pila y van a visitados.
// Completen los dos espacios ____
```

```
function recorridoProfundidad(grafo, inicio) {
  const frontera = [inicio];      // la pila
  const visitados = [];
  while (frontera.length > 0) {
    const nodo = frontera.__( );   // 1. sale el ÚLTIMO de la pila
    if (___) {                     // 2. ¿ya está en visitados?
      continue;
    }
    visitados.push(nodo);
    for (const vecino of grafo[nodo]) {
      if (!visitados.includes(vecino)) {
        frontera.push(vecino);
      }
    }
  }
  return visitados;
}
```

P2 · De Arco a Mercado, con pila *fácil*

El mismo viaje del ejercicio A2, de **Arco** a **Mercado**, ahora por profundidad. Al final comparen los dos caminos.

Datos: grafoPuno (página 1)

a) A mano. Escriban los nombres completos o sus tres primeras letras. Si sale uno que ya está en visitados, escriban su nombre: esa salida se tacha y visitados no cambia. Escriban la pila con el fondo a la izquierda y el tope a la derecha. En la fila de la meta, en «Frontera después» va lo que queda (o «meta»). Si sobran filas, déjenlas en blanco.

N.º	Sale	Pila después	Visitados
1			
2			
3			
4			
5			

Camino que encuentra _____

¿Cuál camino tiene menos aristas? _____

b) En código. Completen la función *dfs* y pruébenla en *Practicas_busqueda.html*.

```
// Profundidad desde inicio hasta meta. Devuelve { camino, orden }.
// Es su bfs del ejercicio A2 con UN cambio. Complétenlo.
function dfs(grafo, inicio, meta) {
  const frontera = [[inicio, null]]; // la pila: pares [nodo, quién lo trajo]
  const visitados = [];
  const padre = {};
  while (frontera.length > 0) {
    const [nodo, previo] = ____; // 1. sale el último
    if (___) {                     // 2. repetido: se tacha
      continue;
    }
    visitados.push(nodo);
    padre[nodo] = previo;
    if (nodo === meta) {
      return { camino: reconstruir(padre, meta), orden: visitados };
    }
    // 3. entran los vecinos no visitados, cada uno con quién lo trajo
  }
  return { camino: null, orden: visitados };
}

function reconstruir(padre, meta) {
  const camino = [meta];
  while (padre[camino[0]] !== null) {
    camino.unshift(padre[camino[0]]);
  }
  return camino;
}
```

P3 · La meta al lado *intermedio*

De **Mercado** a **Muelle**, que son vecinos. Amplitud llega en 3 salidas. Hagan profundidad y cuenten cuántas necesita.

En el código escriban dfs entera, sin espacios para completar.

Datos: grafoPuno (página 1)

a) A mano. Escriban los nombres completos o sus tres primeras letras. Si sale uno que ya está en visitados, escriban su nombre: esa salida se tacha y visitados no cambia. Escriban la pila con el fondo a la izquierda y el tope a la derecha. En la fila de la meta, en «Frontera después» va lo que queda (o «meta»). Si sobran filas, déjenlas en blanco.

N.º	Sale	Pila después	Visitados
1			
2			
3			
4			
5			
6			
7			
8			
9			

¿Cuántas salidas necesitó (contando la tachada)? _____

Camino que encuentra _____

b) En código. Completen la función dfs y pruébenla en Practicas_busqueda.html.

```
// Profundidad desde inicio hasta meta. Devuelve { camino, orden }.
// Si no llega a la meta: { camino: null, orden: ... }
// Las mismas cuatro reglas de bfs; la pila sale por el final (pop).
function dfs(grafo, inicio, meta) {

}

function reconstruir(padre, meta) {

}
```

P4 · Sin la lista de visitados *intermedio*

¿Para qué sirve visitados? Hagan profundidad desde **Huajsapata sin** visitados: sale uno y entran todos sus vecinos, aunque ya hayan salido antes. Anoten las primeras 7 salidas.

Datos: grafoPuno (página 1)

a) A mano. Sin visitados no se tacha nada. La pila con el tope a la derecha.

N.º	Sale	Pila después
1		
2		
3		
4		
5		
6		
7		

¿Qué dos nodos se repiten para siempre? _____

¿Llegaría alguna vez a Muelle? _____

b) En código. Completen la función profundidadSinVisitados y pruébenla en Practicas_busqueda.html.

```
// Profundidad SIN visitados: devuelve las primeras n salidas.
// Ojo: sin visitados el ciclo no termina solo. Paren cuando salidas tenga n elementos.
function profundidadSinVisitados(grafo, inicio, n) {
  const pila = [inicio];
  const salidas = [];
  while (pila.length > 0 && salidas.length < n) {
    // sale el último de la pila, se anota en salidas,
    // y entran TODOS sus vecinos (sin preguntar nada)

  }
  return salidas;
}
```

P5 · El mapa, con pila reto

El mapa del ejercicio A4, de S (A1) a M (C3), ahora por profundidad. Anoten las primeras 8 salidas y después sigan en borrador hasta la meta para escribir el camino.

Datos: mapa1 (página 1)

a) A mano. Vecinos en el orden arriba, abajo, izquierda, derecha: entran en ese orden, así que sale primero el de la derecha.

N.º	Sale	Pila después	Visitados
1			
2			
3			
4			
5			
6			
7			
8			

Camino que encuentra _____

¿Cuántos pasos tiene? (amplitud encontró uno de 4) _____

b) En código. Completen la función `dfsMapa` y pruébenla en `Practicas_busqueda.html`.

```
// Profundidad en un mapa, de S a M. Devuelve { camino, orden }
// con los nombres de las casillas ("A1", "B1"...). Si no llega, camino es null.
// Pueden usar su mapaAGrafo (A4) y su dfs (P3).
function dfsMapa(mapa) {

}
```